

BIT4 — INSTRUCTION SET

Complete ISA Reference | Last updated: 27.06.2026

Overview

BIT4 is a 4-bit stack-based fantasy computer. Each instruction is 8 bits (2 nibbles). Most operations work via two registers (AX, BX) and a stack. Memory operations target a 16-cell RAM in the active bank (16 banks total). Memory-mapped I/O ports occupy addresses 0x00–0x0F in bank 0; STORE there talks to the VPU/system, LOAD reads back state.

ALU operations (stack-based)

Mnemonic	Machine code	Description
AND	0000xxxx	Pop two values, push (a AND b)
OR	0001xxxx	Pop two values, push (a OR b)
XOR	0010xxxx	Pop two values, push (a XOR b)
NOT	0011xxxx	Pop one value, push bitwise NOT
ADD	0100xxxx	Pop two values, push (a + b) mod 16
SUB	0101xxxx	Pop two values, push (a – b) mod 16
SHL	1000xxxx	Pop one, shift left 1, push
SHR	1001xxxx	Pop one, shift right 1, push
INC	1010xxxx	Pop one, +1, push
DEC	1011xxxx	Pop one, –1, push

Register / stack operations

Mnemonic	Machine code	Description
MOV AX,n	1100nnnn	Load nibble n into AX (immediate)
MOV BX,n	1101nnnn	Load nibble n into BX (immediate)
MOV AX,BX	11001101	Copy BX → AX
MOV BX,AX	11011100	Copy AX → BX
PUSH AX	1100xxxx	Push AX onto stack (after MOV pattern)
PUSH BX	1101xxxx	Push BX onto stack
POP AX	1110xxxx	Pop top of stack into AX
POP BX	1111xxxx	Pop top of stack into BX

Memory operations

Mnemonic	Machine code	Description
LOAD addr	0110aaaa	Push contents of RAM[addr] in active bank
STORE addr	0111aaaa	Pop value, write to RAM[addr] in active bank
LOADB bank,addr	01101111	Push RAM[bank,addr] (operand follows)
STOREB bank,addr	01111111	Pop value, write to RAM[bank,addr]
LOADBI bank,reg	01101110	Indirect: push RAM[bank, value-of-reg]
STOREBI bank,reg	01111110	Indirect: pop, write to RAM[bank, value-of-reg]

Mnemonic	Machine code	Description
LOADBI+ b,r	01101111	Same as LOADBI then increment reg (auto-advance)
STOREBI+ b,r	01111111	Same as STOREBI then increment reg (auto-advance)

Control flow

Mnemonic	Machine code	Description
NOP	00000001	No operation
HALT	11111110	Stop execution (marker)
JMP lbl	11111111	Unconditional jump to label
JZ lbl	11111110	Jump if zero flag set
JNZ lbl	11111011	Jump if zero flag clear
CALL lbl	11111101	Push return address, jump to label
RET	11111100	Pop return address, jump back

Compare instructions

Mnemonic	Machine code	Description
CMP AX,n	10111100	Set zero flag if AX == n (immediate)
CMP BX,n	10111101	Set zero flag if BX == n
CMP AX,BX	10111110	Set zero flag if AX == BX
CMP BX,AX	10111111	Set zero flag if BX == AX

VPU / I/O instructions

Mnemonic	Machine code	Description
PRINT	11110111	Print char (ROM page+ID on stack) at cursor

Most VPU work is done via STORE to ports 0x00–0x0F in bank 0 (see Port Map document). PRINT is a dedicated opcode because it composes ROM lookup, color attribute, and cursor advance in one step.

Notational conventions

- **nnnn / aaaa / xxxx** — variable nibble (depends on operand).
- **x** in low nibble — "don't care" / determined by stack at runtime.
- Some opcodes share codes (e.g. HALT and JZ both 11111110) because the engine disambiguates by mnemonic in the source text; this is part of the current label-based execution model and will resolve to distinct codes when the project moves to address-based jumps.
- All immediate values are 4-bit (0–15).
- Memory addresses are 4-bit (0–15) within a bank; 16 banks total.

BIT4 is a hobby / educational fantasy 4-bit computer. See the companion Port Map document for memory-mapped I/O details.